🚀 Lecture 04.04: Lists & Dynamic Array Memory Architecture

📌 Executive Architectural Summary

A Python `list` is a **mutable, ordered sequence of heterogeneous object references**. Internally in CPython (`PyListObject`), a list is implemented not as a linked list, but as a **contiguous array of pointers** (`PyObject** items`) to objects scattered across the heap.

🔬 Core Engineering Mechanics

1. Pointer Array Architecture

```
PyListObject Header
- ob_refcnt
- ob_size = 3
- allocated = 4
- ob_item ------> [ Pointer 0 ] ------> PyLongObject(10)
                  [ Pointer 1 ] ------> PyUnicodeObject("AI")
                  [ Pointer 2 ] ------> PyFloatObject(3.14)
                  [ Empty Slot ]
```

Because the list only stores 8-byte pointer addresses, it can hold heterogeneous data types while maintaining $O(1)$ random access time by index.

2. Mutability Invariant

Unlike strings and tuples, lists are mutable. Elements can be reassigned, inserted, or removed in-place without altering the memory address (`id()`) of the list itself.

💻 Production Python Implementation

```
import sys

# 1. Heterogeneous List Creation
features = [101, "Convolutional", 3.1415, True]
print(f"List: {features} (Length: {len(features)})")
```

```
# 2. In-Place Mutability
print(f"Before Modification ID: {id(features)}")
features[0] = 999
features.append("DenseLayer")
print(f"After Modification ID:  {id(features)} (Same object!)")
print(f"Updated List: {features}")
```